

Oyster Mushroom Cultivation Expert System

Project Overview

Specialisation. The system diagnoses common cultivation problems in oyster mushroom (*Pleurotus* spp.) production — environmental stress, contamination/pest issues, and pinning/yield failures — from the growing conditions and symptoms a grower reports.

Problem it solves. Oyster mushroom growing in Sri Lanka is mostly a small-scale, cottage-level enterprise, and most cultivators fail to meet demand and quality because they lack the technical knowledge to interpret abnormal growth or disease symptoms and cannot always reach a specialist. This system packages reliable extension and research knowledge into an always-available diagnostic advisor.

Decisions/output provided. For each consultation the system returns, for every problem it can support with evidence:

1. The possible cultivation problem (e.g. *high CO₂ stress*, *green mould contamination*).
2. Supporting reasoning — the specific symptoms/conditions that were matched.
3. The rule(s) applied (rule IDs, traceable to the Rules section below).
4. A management/corrective recommendation.

Scope. In scope: oyster mushroom cultivation (not general mushroom biology or taxonomy); environmental/growing problems (temperature, humidity, CO₂, light, airflow); common contamination and pest indicators; pinning and yield problems; management recommendations grounded in HORDI, UF/IFAS and peer-reviewed *Pleurotus* research. Out of scope: field identification of wild mushroom species, food-safety/toxicology, and business/financial planning.

Architecture. Next.js + React + Tailwind (form and results UI) → FastAPI (turns the UI request into Prolog facts and calls the engine) → SWI-Prolog (the expert system itself: facts, rules, forward- and backward-chaining inference, and explanation generation). The reasoning lives entirely in Prolog; the other two layers are presentation and plumbing.

Expert System Components

This project maps onto the five standard components of a rule-based expert system:

1. **User Interface** — allows the mushroom grower to enter observed symptoms and environmental conditions.
2. **Knowledge Base** — contains the domain facts (cultivation thresholds, disease information, recommendations) and the diagnostic rules.
3. **Inference Engine** — uses backward chaining and forward chaining to derive possible cultivation problems from the grower's input.
4. **Explanation Facility** — shows the evidence and the rule responsible for each diagnosis.
5. **Knowledge Engineer** — the developer responsible for collecting, validating and encoding the domain knowledge.

System Architecture

The course lecture's generic expert-system architecture — **User ↔ User Interface ↔ Inference Engine ↔ Knowledge Base ↔ Knowledge Editor ↔ Knowledge Engineer**, connected by facts/results, knowledge and updates — maps directly onto this project's five architectural layers plus the person who built the knowledge base:

Facts flow up to the knowledge base; recommendations flow back down

Standard expert-system architecture, mapped onto this project's components

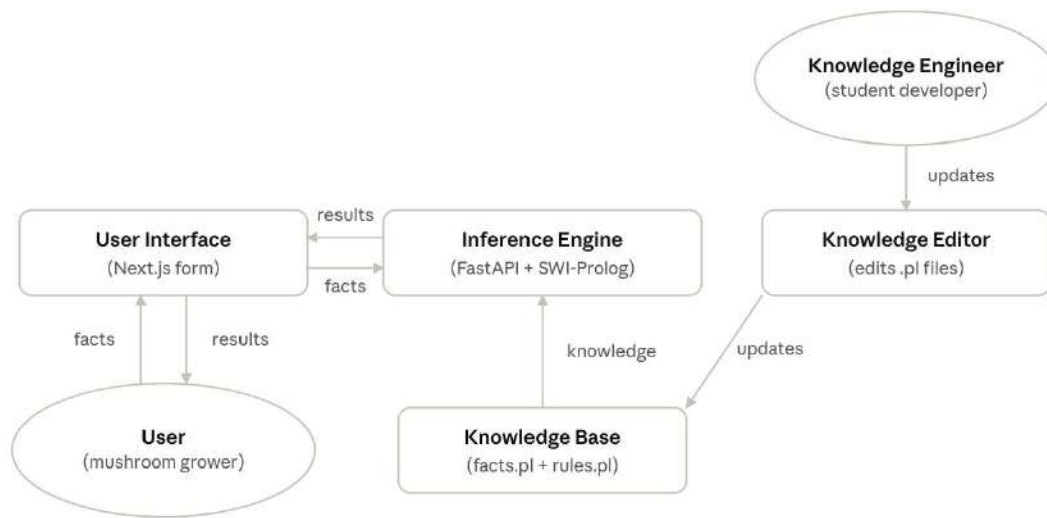


Figure 1: Expert-system architecture — six generic roles mapped onto this project's five layers.

- **User** — the mushroom grower running a consultation.
- **User Interface** — the Next.js/React consultation form and results page (frontend/).
- **Inference Engine** — FastAPI plus SWI-Prolog: FastAPI turns the submitted form into Prolog case facts and invokes backward chaining (`diagnose/1`, SLD resolution) or forward chaining (`forward_chain/0`, fixpoint iteration) over the same rule base.
- **Knowledge Base** — the Prolog fact and rule base (`prolog/facts.pl`, `prolog/rules.pl`): 32 domain facts and 24 diagnostic rules (see Predicates, Facts and Rules below).
- **Knowledge Editor** — no separate GUI editor was built; the knowledge engineer edits `prolog/facts.pl` and `prolog/rules.pl` directly, which is the "editor" role in this implementation.
- **Knowledge Engineer** — the student developer who acquired the domain knowledge (HORDI, UF/IFAS, peer-reviewed *Pleurotus* research) and encoded it as predicates, facts and rules (see Knowledge Engineering Process below).

Facts and results travel the User → Interface → Inference Engine leg on every consultation; knowledge travels the Inference Engine → Knowledge Base leg on every query; updates travel the Knowledge Engineer → Knowledge Editor → Knowledge Base leg only when the rule base itself changes (during development, not at grower run-time). The detailed six-step runtime pipeline for a single consultation is diagrammed separately in Relationships & Inference Strategy below.

Choice of Prolog

SWI-Prolog was selected as the expert-system engine because Prolog is designed for symbolic reasoning using facts, rules and logical queries. Its native SLD resolution naturally supports backward chaining, while a separate rule-firing mechanism (`forward_chain/0`) was implemented over the same rule/3 facts to demonstrate forward chaining as required by the assignment. Prolog also keeps the diagnostic rules explicit and readable as `diagnose/1` clauses, so the knowledge base stays easy to inspect, verify against its sources, and extend with new rules.

Why Next.js + FastAPI + Prolog?

The system is a three-tier design rather than a single Prolog program so that each tier does only the job it is suited to:

Component	Responsibility
Next.js / React	User interface — the consultation form and the results view
Tailwind CSS	UI styling
FastAPI	API communication, request validation, and translating form input into Prolog facts
SWI-Prolog	Knowledge representation and reasoning
Prolog facts/rules	Expert knowledge — the domain facts and the 24 diagnostic rules
Prolog inference engine	Backward- and forward-chaining reasoning over that knowledge

The diagnostic intelligence is implemented in Prolog rather than in the frontend or Python backend.

Next.js only renders what SWI-Prolog concludes, and FastAPI only validates and shuttles data between the two, so every diagnosis's evidence, rules-applied list and explanation text originate on the Prolog side (see Input and Output Representation).

System Interface

The system exposes one interface: a single-page Next.js web form (`frontend/app/page.tsx`). There is no separate knowledge-editor UI — as noted above, rule and fact changes are made directly in the Prolog source files.

Consultation form (input). The grower optionally sets the growth stage and days since spawning, ticks any observed symptoms from a checklist, and fills in whichever growing-condition, hygiene and room fields they can report — every field is optional, and CO₂ can be given as a category (High/Fluctuating/Normal) or a ppm reading.

The image shows a web application interface for a consultation form, running on localhost:3000. It is divided into three main sections:

- 01 Growth details:** Includes a dropdown for 'Growth stage' set to 'Spawn run (incubation)' and a text input for 'Days since spawning' set to '14'.
- 02 Observed symptoms:** A section titled 'Tap all that apply' with multiple toggle buttons. The buttons 'Long stem' and 'Green patches on substrate' are highlighted in orange, indicating they are selected. Other buttons include 'Small / underdeveloped cap', 'Pale cap colour', 'Brittle cap', 'Dry, cracked cap', 'Pins drying out', 'Pins aborting before maturity', 'Long, thin stem', 'No cap forming', 'Poor air circulation', 'Green sporulation', 'Brown spots on cap', 'Insects present', 'No mycelium growth', 'Patchy colonisation', 'Thin stipe', 'Delayed fruiting-body formation', and 'Abnormal fruiting-body shape'.
- 03 Growing conditions:** A section with various inputs:
 - 'CO2 level (category)' dropdown set to 'fluctuating'.
 - 'CO2 reading (ppm) (ppm)' text input set to '0'.
 - 'Light hours per day (hours/day)' dropdown set to 'not measured'.
 - 'Relative humidity (%) (%)' text input set to 'not measured'.
 - 'Temperature drop offered to trigger pinning' dropdown set to '-- not reported --'.
 - 'Fresh-air exchange quality' dropdown set to 'poor'.
 - 'Substrate colonisation' dropdown set to '-- not reported --'.
 - 'Spawn age' dropdown set to '-- not reported --'.
 - 'Spawn source' text input.
 - 'Free water sitting on caps' text input.

Figure 2: Consultation form filled in with observed symptoms and growing conditions.

Results view (output). Each matched problem is rendered as its own card: the problem name, the supporting evidence (which symptoms/conditions matched), the rule(s) applied, a plain-language explanation, and a management recommendation — so a grower sees not just *what* the system concluded but *why* (see Input and Output Representation below for the underlying JSON contract).

No diagnosis / unknown case. If the available evidence does not match any rule in the current knowledge base, the system does not force a diagnosis — the empty-result case is a deliberate, tested outcome (see Testing), not a failure. Because *no diagnosis* is not the same as *no problem*, the results view tells the grower plainly that no supported problem was identified and invites them to enter additional symptoms or measurements, rather than showing nothing.

The image shows a 'Results' view with a green checkmark icon and the text 'forward chaining'. Below this is a card for 'Green Mold Contamination' with a small 'R14' label in the top right corner.

SUPPORTING REASONING

- green_patches_on_substrate was observed
- 14 days have passed since spawning

EXPLANATION

Because green_patches_on_substrate was observed; 14 days have passed since spawning, rule(s) R14 concluded: Green Mold Contamination.

MANAGEMENT RECOMMENDATION

Remove the affected bag away from the growing room; re-sterilise equipment and improve substrate sterilisation for the next batch.

Figure 3: Results view — a diagnosed problem card with evidence, explanation and recommendation (Green Mold Contamination, rule R14, forward chaining).

Knowledge Engineering Process

The system was built by following the standard knowledge-engineering pipeline end to end:

Domain Selection → oyster mushroom (*Pleurotus* spp.) cultivation problem diagnosis, chosen because it is a well-bounded, practically useful domain with strong extension-service and peer-reviewed documentation.

Domain Study → HORDI/Dept. of Agriculture Sri Lanka cultivation guides, UF/IFAS extension publications, and peer-reviewed *Pleurotus* research were read to understand cultivation stages, environmental requirements, and common failure modes (see Sources).

Knowledge Acquisition → 24 discrete pieces of knowledge (KA01–KA24) were extracted from those sources and logged in the Knowledge Acquisition Table below, each tagged with where it came from and how it would be used.

Knowledge Representation → the acquired knowledge was organised into the predicate schema in the Predicates section: domain facts, per-consultation case facts, and derived (inferred) facts.

Facts & Rules → 32 domain facts and 24 diagnostic rules were written in SWI-Prolog from the acquired knowledge (Facts and Rules sections); each rule cites the KA id(s) it is grounded in.

Inference → both backward chaining (diagnose/1, SLD resolution from a query) and forward chaining (forward_chain/0, fixpoint iteration over all rules) were implemented over the same rule/3 fact base, so the system demonstrates both required reasoning strategies (Relationships & Inference Strategy section).

Testing → the rule base and API were tested with 56 automated tests (40 Prolog PIUnit tests + 16 FastAPI integration tests) covering every rule individually, combinations of rules, and edge cases (Testing section).

Explanation → every diagnosis carries its evidence, the specific rule id(s) that fired, a plain-language explanation sentence, and a recommendation — never a bare answer (see Input and Output Representation, and the User Input → Facts → Rules Applied → Reasoning → Final Conclusion pipeline diagram there).

Knowledge Acquisition Table

ID	Knowledge	Source	How it will be used
KA01	Substrate must contain lignin, cellulose and hemicellulose (e.g. sawdust, paddy straw); preparation is simple and doesn't need tightly controlled conditions	HORDI, Dept. of Agriculture Sri Lanka	Background substrate_component/1 facts
KA02	Oyster mushroom tolerates a wider range of temperature, humidity and CO ₂ than most cultivated mushrooms, but still needs suitable conditions for a consistent crop	HORDI	Basis for threshold/range facts (optimal_condition/5) rather than rigid pass/fail checks
KA03	Cultivation proceeds through spawn preparation → substrate preparation/sterilisation → spawning	HORDI	Basis for growth_stage/1 and

ID	Knowledge	Source	How it will be used
	→ spawn run (incubation room) → primordia/fruiting body formation (cropping house)		stage-scoped rules
KA04	Released Sri Lankan varieties include American oyster (white), Abalone (light greyish-brown) and Pink oyster, e.g. the "Makandura White" release	HORDI	variety/2 background facts
KA05	Spawn run is commonly carried out around 24–28°C in darkness	Cultivation process literature; HORDI practice	optimal_condition(spawn_run, temperature, 24, 28, celsius)
KA06	A ~5°C drop from colonisation temperature toward fruiting temperature reliably triggers pinning	<i>Pleurotus</i> /mushroom cultivation guides	Rule for delayed_pinning when no temperature drop is reported
KA07	Fruiting-stage relative humidity should be ~80–90%; close to 90% is needed to initiate pinning	<i>Pleurotus</i> research; extension guides	optimal_condition(fruiting, humidity, 80, 90, percent); pinning-humidity rule
KA08	CO ₂ should stay under ~1200 ppm during fruiting; above that, fruiting is inhibited	<i>Pleurotus</i> production research (MDPI); grower guides	co2_threshold(fruiting, 1200); high-CO ₂ rule
KA09	Poor fresh-air exchange (high CO ₂) produces long stems with small caps — classic "leggy" fruiting bodies	Multiple grower/technical sources	Core symptom rule for high_co2_stress
KA10	Insufficient light also produces long, leggy stems and small, pale caps, and is easily confused with high CO ₂	Grower guides	Rule distinguishing insufficient_light from high_co2_stress
KA11	Oyster mushroom needs indirect, "reading-level" light during fruiting (roughly 12–18 h/day), unlike button mushroom which fruits in darkness	Production research; grower guides	photoperiod(fruiting, 12, 18); light-deficiency rule
KA12	No pins after ~14 days under otherwise correct conditions usually means CO ₂ still too high, or incomplete colonisation	Cultivation guide	Two-branch rule for no_pinning
KA13	Humidity without fresh-air exchange gives long, spindly stems and no caps; fresh-air exchange without enough humidity dries out forming pins	Cultivation guide	Rule distinguishing airflow failure from humidity failure

ID	Knowledge	Source	How it will be used
	— both are needed together		
KA14	Formed pins can abort before maturity when CO ₂ fluctuates or humidity is inconsistent; aborted pins look small, dark/dense and stop developing	Grower/technical sources	Rule for pin_abortion
KA15	Dry air causes pins to dry out, brown/bronze at the edges, with curling caps	Grower guide	Rule for low_humidity_stress
KA16	Green mould disease is caused by <i>Trichoderma</i> spp.; shows as green sporulation on the substrate, typically 10–15 days after spawning; severe cases yield no mushrooms	Peer-reviewed review (ScienceDirect); disease review	Rule for green_mold_contamination
KA17	Contamination is commonly traced to improperly sterilised substrate, poor hygiene, unsterilised equipment, or spent substrate left near the growing room	Grower guide	Rule linking poor hygiene_practice/2 facts to contamination
KA18	Bacterial Brown Blotch Disease (<i>Pseudomonas tolaasii</i>) develops more under cool, high-humidity/high-rainfall conditions and from contaminated irrigation or airborne contamination	Peer-reviewed source (PMC/NCBI)	Rule for bacterial_brown_blotch
KA19	Insects/pests mainly enter cultivation rooms through doors/windows without wire mesh screening	Grower guide	Rule for pest_infestation
KA20	Spawn quality directly determines yield; spawn should be fresh and from a reputable source — old or poor-quality spawn causes weak or absent colonisation	Grower guide	Rule for poor_spawn_quality
KA21	Overcrowded bags/blocks reduce air circulation and raise contamination risk	Grower guide	Rule for overcrowding
KA22	Viral infection can cause delayed fruiting-body formation, a shortened stipe, abnormal shape and thin caps, resembling "La France" disease in button mushroom	Peer-reviewed review (ScienceDirect)	Lower-confidence rule for a possible viral-related abnormality (possible_viral_infection), reached via negation as failure only once other causes are ruled out
KA23	Oyster mushroom growing in Sri Lanka is mostly a household/cottage business, and most cultivators fail to meet demand or quality due to	Sri Lanka grower resource	Motivates the Project Overview /

ID	Knowledge	Source	How it will be used
	limited technical knowledge		problem statement
KA24	Of temperature, humidity and CO ₂ , temperature alone has the smallest effect on fruiting-body dimensions (mainly cap height); humidity and CO ₂ matter more	Peer-reviewed research (2018)	Justifies weighting CO ₂ /humidity evidence more heavily than temperature alone

Together, the **Predicates**, **Facts** and **Rules** sections below define the system's **Knowledge Base**; the **Inference Engine** that reasons over it is described afterward, in **Relationships & Inference Strategy**.

Knowledge Representation

The diagram below sets out the expert-system concepts on their own, separate from the concrete System Architecture above: the two knowledge-base components (Facts and Rules) feed a single inference engine that can reason in either direction, and that reasoning is turned in turn into a diagnosis, an explanation and a recommendation.

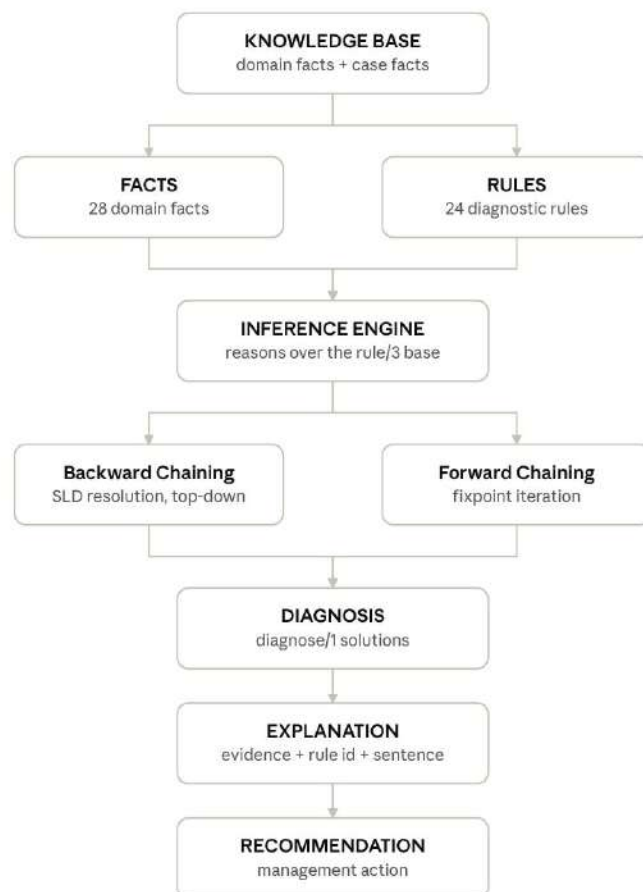


Figure 4: Knowledge representation — facts and rules feeding a single inference engine that yields a diagnosis, explanation and recommendation.

Predicates

The knowledge base uses three families of predicates: **domain knowledge** (loaded once — the "textbook" the system was built from), **case knowledge** (asserted per consultation, from the grower's input) and **derived knowledge** (produced by inference, used to build the explanation).

Domain knowledge (static facts)

- `optimal_condition(Stage, Parameter, Min, Max, Unit)` — the ideal range of an environmental parameter for a growth stage, e.g. `optimal_condition(fruiting, humidity, 80, 90, percent)`.
- `co2_threshold(Stage, MaxPPM)` — the CO₂ ceiling above which a stage is negatively affected.
- `photoperiod(Stage, MinHours, MaxHours)` — recommended daily light hours for a stage.
- `substrate_component(Material)` — accepted lignocellulosic substrate materials.
- `variety(Name, ColourDescription)` — known Sri Lankan oyster varieties.
- `disease_cause(Problem, CausalAgent)` — the pathogen/agent behind a contamination problem.
- `disease_symptom(Problem, Symptom)` — a recognised symptom of a named problem.
- `recommendation(Problem, Action)` — the management/corrective action for a diagnosed problem.
- `rule(RuleID, ConditionList, Conclusion)` — a data representation of every diagnostic rule, used to drive forward chaining (see Relationships & Inference Strategy).

Case knowledge (asserted per consultation)

- `observed_symptom(Symptom)` — a symptom the grower reports, e.g. `observed_symptom(long_stem)`.
- `current_condition(Parameter, Value)` — a measured or estimated condition, e.g. `current_condition(co2_level, high)`.
- `growth_stage(Stage)` — the crop's current stage (`spawn_run`, `pinning` or `fruiting`).
- `days_since(Event, Days)` — elapsed time since an event, e.g. `days_since(spawning, 16)`.
- `hygiene_practice(Practice, Status)` — hygiene-related facts, e.g. `hygiene_practice(substrate_sterilization, poor)`.
- `room_condition(Feature, Status)` — physical room facts, e.g. `room_condition(window_screens, absent)`.

Derived knowledge (produced by inference)

- `diagnose(Problem)` — a problem the engine concludes is present.
- `evidence_for(Problem, Fact)` — one fact that supported a diagnosis; collected into the reasoning trace.
- `rule_fired(Problem, RuleID)` — which named rule produced the diagnosis.
- `explanation(Problem, Text)` — the natural-language reasoning sentence generated for the grower.

Facts

32 static domain facts are defined, covering environmental thresholds, substrate, varieties, disease agents/symptoms and recommendations (see the Knowledge Acquisition Table for sourcing) — exceeding the assignment's minimum requirement of 20 facts.

Environmental thresholds (7)

- optimal_condition(spawn_run, temperature, 24, 28, celsius).
- optimal_condition(fruiting, temperature, 20, 25, celsius).
- optimal_condition(pinning, humidity, 85, 95, percent).
- optimal_condition(fruiting, humidity, 80, 90, percent).
- co2_threshold(fruiting, 1200).
- co2_threshold(pinning, 1000).
- photoperiod(fruiting, 12, 18).

Substrate (4)

- substrate_component(sawdust).
- substrate_component(paddy_straw).
- substrate_component(sugarcane_bagasse).
- substrate_component(cotton_waste).

Varieties (3)

- variety(american_oyster, white).
- variety(abalone, light_greyish_brown).
- variety(pink_oyster, pink).

Disease causal agents (2)

- disease_cause(green_mold_contamination, trichoderma_spp).
- disease_cause(bacterial_brown_blotch, pseudomonas_tolaasii).

Disease symptoms (2)

- disease_symptom(green_mold_contamination, green_patches_on_substrate).
- disease_symptom(bacterial_brown_blotch, brown_spots_on_cap).

Recommendations (14)

- recommendation(high_co2_stress, 'Increase fresh-air exchange; fully change the air in the fruiting room every 8-10 minutes').
- recommendation(insufficient_light, 'Provide 12-18 hours/day of indirect fluorescent or LED light at reading intensity').
- recommendation(low_humidity_stress, 'Increase misting frequency and use a humidity tent; maintain 80-90% relative humidity').
- recommendation(green_mold_contamination, 'Remove the affected bag away from the growing room; re-sterilise equipment and improve substrate sterilisation for the next batch').
- recommendation(bacterial_brown_blotch, 'Reduce free water on caps, improve air movement, avoid late-day overhead misting, and use clean irrigation water').
- recommendation(pest_infestation, 'Fit wire mesh screens on all doors and windows and improve room sanitation').
- recommendation(poor_spawn_quality, 'Source fresh spawn from a reputable supplier and discard old or discoloured spawn').
- recommendation(overcrowding, 'Reduce the number of bags per shelf/unit area to restore air circulation').

- recommendation(no_pinning, 'Verify full colonisation, strengthen fresh-air exchange to lower CO₂, and ensure a ~5C drop to fruiting temperature').
- recommendation(pin_abortion, 'Stabilise CO₂ and humidity; avoid fluctuating fresh-air exchange during pin development').
- recommendation(delayed_pinning, 'Move blocks to a fruiting space roughly 5C cooler than the colonisation room to trigger pinning').
- recommendation(airflow_without_humidity, 'Add fresh-air exchange AND maintain 80-90% humidity together; one without the other still stalls fruiting').
- recommendation(general_contamination, 'Re-sterilise substrate and equipment, dispose of spent substrate away from the growing room, and tighten hygiene practices').
- recommendation(possible_viral_infection, 'Isolate and destroy affected blocks; source certified virus-free spawn for the next batch and disinfect the growing room').

The table below lists every fact by its predicate id (F01–F32), matching the grouping above; it is provided so the knowledge base can be checked item by item against the Knowledge Acquisition Table.

No.	Predicate	Example	Purpose
F01	optimal_condition/5	optimal_condition(spawn_run, temperature, 24, 28, celsius)	Environmental requirement — spawn-run temperature
F02	optimal_condition/5	optimal_condition(fruiting, temperature, 20, 25, celsius)	Environmental requirement — fruiting temperature
F03	optimal_condition/5	optimal_condition(pinning, humidity, 85, 95, percent)	Environmental requirement — pinning humidity
F04	optimal_condition/5	optimal_condition(fruiting, humidity, 80, 90, percent)	Environmental requirement — fruiting humidity
F05	co2_threshold/2	co2_threshold(fruiting, 1200)	CO ₂ ceiling for fruiting
F06	co2_threshold/2	co2_threshold(pinning, 1000)	CO ₂ ceiling for pinning
F07	photoperiod/3	photoperiod(fruiting, 12, 18)	Light requirement for fruiting
F08	substrate_component/1	substrate_component(sawdust)	Accepted substrate material
F09	substrate_component/1	substrate_component(paddy_straw)	Accepted substrate material
F10	substrate_component/1	substrate_component(sugarcane_bagasse)	Accepted substrate material
F11	substrate_component/1	substrate_component(cotton_waste)	Accepted substrate material
F12	variety/2	variety(american_oyster, white)	Known Sri Lankan cultivar
F13	variety/2	variety(abalone, light_greyish_brown)	Known Sri Lankan cultivar

No.	Predicate	Example	Purpose
F14	variety/2	variety(pink_oyster, pink)	Known Sri Lankan cultivar
F15	disease_cause/2	disease_cause(green_mold_contamination, trichoderma_spp)	Causal agent
F16	disease_cause/2	disease_cause(bacterial_brown_blotch, pseudomonas_tolaasii)	Causal agent
F17	disease_symptom/2	disease_symptom(green_mold_contamination, green_patches_on_substrate)	Recognised symptom
F18	disease_symptom/2	disease_symptom(bacterial_brown_blotch, brown_spots_on_cap)	Recognised symptom
F19	recommendation/2	recommendation(high_co2_stress, '...')	Corrective action
F20	recommendation/2	recommendation(insufficient_light, '...')	Corrective action
F21	recommendation/2	recommendation(low_humidity_stress, '...')	Corrective action
F22	recommendation/2	recommendation(green_mold_contamination, '...')	Corrective action
F23	recommendation/2	recommendation(bacterial_brown_blotch, '...')	Corrective action
F24	recommendation/2	recommendation(pest_infestation, '...')	Corrective action
F25	recommendation/2	recommendation(poor_spawn_quality, '...')	Corrective action
F26	recommendation/2	recommendation(overcrowding, '...')	Corrective action
F27	recommendation/2	recommendation(no_pinning, '...')	Corrective action
F28	recommendation/2	recommendation(pin_abortion, '...')	Corrective action
F29	recommendation/2	recommendation(delayed_pinning, '...')	Corrective action
F30	recommendation/2	recommendation(airflow_without_humidity, '...')	Corrective action

No.	Predicate	Example	Purpose
F31	recommendation/2	recommendation(general_contamination, '...')	Corrective action
F32	recommendation/2	recommendation(possible_viral_infection, '...')	Corrective action

Rules

24 rules are defined as diagnose/1 clauses (backward-chaining form). Each is also representable as a rule(RuleID, Conditions, Conclusion) fact for the forward-chaining engine (see Relationships & Inference Strategy) — exceeding the assignment's minimum requirement of 20 rules.

A. Environmental / airflow / light problems

- **R1** diagnose(high_co2_stress) :- observed_symptom(long_stem), observed_symptom(small_cap), current_condition(co2_level, high). — the classic "leggy" signature (KA09).
- **R2** diagnose(high_co2_stress) :- growth_stage(Stage), current_condition(co2_ppm, PPM), co2_threshold(Stage, Max), PPM > Max. — numeric-threshold alternative when a PPM reading is available (KA08).
- **R3** diagnose(insufficient_light) :- observed_symptom(long_stem), observed_symptom(pale_cap), current_condition(light_hours, Hours), growth_stage(fruiting), photoperiod(fruiting, Min, _), Hours < Min. — distinguishes light deficiency from CO₂ stress via pale colour and light hours (KA10, KA11).
- **R4** diagnose(insufficient_light) :- observed_symptom(brittle_cap), current_condition(light_hours, Hours), Hours =< 4. — (KA11).
- **R5** diagnose(low_humidity_stress) :- observed_symptom(dry_cracked_cap), growth_stage(fruiting), current_condition(humidity_percent, H), optimal_condition(fruiting, humidity, Min, _, percent), H < Min. — (KA07, KA15).
- **R6** diagnose(low_humidity_stress) :- observed_symptom(pins_drying), current_condition(humidity_percent, H), H < 70. — (KA15).
- **R7** diagnose(no_pinning) :- growth_stage(fruiting), days_since(spawning, D), D >= 14, current_condition(co2_level, high). — (KA12).
- **R8** diagnose(no_pinning) :- growth_stage(fruiting), days_since(spawning, D), D >= 14, current_condition(colonisation, incomplete). — alternative cause for the same symptom (KA12).
- **R9** diagnose(delayed_pinning) :- growth_stage(fruiting), current_condition(temperature_drop, none). — (KA06).
- **R10** diagnose(pin_abortion) :- observed_symptom(aborted_pins), current_condition(co2_level, fluctuating). — (KA14).
- **R11** diagnose(pin_abortion) :- observed_symptom(aborted_pins), current_condition(humidity_percent, H), H < 75. — (KA14).
- **R12** diagnose(airflow_without_humidity) :- observed_symptom(long_thin_stem), observed_symptom(no_cap), current_condition(fresh_air_exchange, good), current_condition(humidity_percent, H), H < 70. — the two-trigger interaction (KA13).
- **R13** diagnose(overcrowding) :- room_condition(bag_spacing, crowded), observed_symptom(poor_air_circulation). — (KA21).

B. Contamination, disease and pest problems

- **R14** diagnose(green_mold_contamination) :- observed_symptom(green_patches_on_substrate), days_since(spawning, D), $D \geq 10$, $D \leq 15$. — matches the reported onset window (KA16).
- **R15** diagnose(green_mold_contamination) :- observed_symptom(green_sporulation), disease_cause(green_mold_contamination, trichoderma_spp). — (KA16).
- **R16** diagnose(bacterial_brown_blotch) :- observed_symptom(brown_spots_on_cap), current_condition(humidity_percent, H), $H > 90$. — (KA18).
- **R17** diagnose(bacterial_brown_blotch) :- observed_symptom(brown_spots_on_cap), current_condition(free_water_on_caps, present). — (KA18).
- **R18** diagnose(general_contamination) :- hygiene_practice(substrate_sterilization, poor). — (KA17).
- **R19** diagnose(general_contamination) :- hygiene_practice(equipment_sterilization, poor). — (KA17).
- **R20** diagnose(general_contamination) :- room_condition(spent_substrate_disposal, near_growing_room). — (KA17).
- **R21** diagnose(pest_infestation) :- observed_symptom(insects_present), room_condition(window_screens, absent). — (KA19).
- **R22** diagnose(poor_spawn_quality) :- observed_symptom(no_mycelium_growth), current_condition(spawn_age, old). — (KA20).
- **R23** diagnose(poor_spawn_quality) :- observed_symptom(patchy_colonisation), current_condition(spawn_source, unverified). — (KA20).
- **R24** diagnose(possible_viral_infection) :- observed_symptom(thin_stipe), observed_symptom(delayed_fruiting), observed_symptom(abnormal_shape), \+ diagnose(high_co2_stress), \+ diagnose(insufficient_light). — a low-confidence, last-resort rule for a **possible viral-related abnormality** (KA22). The two \+ goals are Prolog's negation as failure: this conclusion is reached only once the system cannot already establish the far more common high-CO₂ or insufficient-light explanations for the same symptoms, so it never overrides a more likely diagnosis.

Rule Validation and Knowledge Sources

The rules in the knowledge base were derived from documented mushroom cultivation and disease-management sources. Each rule is linked to one or more knowledge-acquisition items (KA01–KA24), which identify the source used to obtain the underlying domain knowledge. Therefore, the rules are not arbitrarily created; they are based on documented cultivation guidelines, extension publications, and peer-reviewed research.

The table below cites the source and Knowledge Acquisition id behind every rule; full bibliographic details for each [S#] tag are in Sources.

Rule	Rule description	Source	Knowledge ID
R1	Long stems + small caps + high CO ₂ → high CO ₂ stress	[S13]	KA09
R2	CO ₂ reading above the stage's threshold → high CO ₂ stress	[S10]	KA08
R3	Long stems + pale caps + too few light hours → insufficient light	[S5], [S13]	KA10, KA11

Rule	Rule description	Source	Knowledge ID
R4	Brittle caps + ≤ 4 light hours $\rightarrow$ insufficient light	[S5]	KA11
R5	Dry, cracked caps + humidity below the fruiting minimum $\rightarrow$ low humidity stress	[S9], [S13]	KA07, KA15
R6	Pins drying + humidity $< 70\%$ $\rightarrow$ low humidity stress	[S13]	KA15
R7	No pins after ≥ 14 days + high CO ₂ $\rightarrow$ no pinning	[S15]	KA12
R8	No pins after ≥ 14 days + incomplete colonisation $\rightarrow$ no pinning	[S15]	KA12
R9	No temperature drop reported at fruiting $\rightarrow$ delayed pinning	[S15]	KA06
R10	Aborted pins + fluctuating CO ₂ $\rightarrow$ pin abortion	[S11]	KA14
R11	Aborted pins + humidity $< 75\%$ $\rightarrow$ pin abortion	[S11]	KA14
R12	Good airflow but humidity $< 70\%$, long thin stem, no cap $\rightarrow$ airflow-without-humidity	[S14]	KA13
R13	Crowded bag spacing + poor air circulation $\rightarrow$ overcrowding	[S11]	KA21
R14	Green patches 10–15 days after spawning $\rightarrow$ green mould contamination	[S6]	KA16
R15	Green sporulation + Trichoderma causal link $\rightarrow$ green mould contamination	[S6]	KA16
R16	Brown spots on cap + humidity $> 90\%$ $\rightarrow$ bacterial brown blotch	[S8]	KA18
R17	Brown spots on cap + free water present $\rightarrow$ bacterial brown blotch	[S8]	KA18
R18	Poor substrate sterilisation $\rightarrow$ general contamination	[S7]	KA17
R19	Poor equipment sterilisation $\rightarrow$ general contamination	[S7]	KA17
R20	Spent substrate stored near the growing room $\rightarrow$ general contamination	[S7]	KA17

Rule	Rule description	Source	Knowledge ID
R21	Insects present + window screens absent → pest infestation	[S12]	KA19
R22	No mycelium growth + old spawn → poor spawn quality	[S13]	KA20
R23	Patchy colonisation + unverified spawn source → poor spawn quality	[S13]	KA20
R24	Thin stipe + delayed fruiting + abnormal shape, once the common causes are ruled out → possible viral-related abnormality	[S6]	KA22

Relationships & Inference Strategy

Domain facts describe the expert's static knowledge (optimal ranges, thresholds, disease causes, recommendations); case facts describe one grower's consultation (symptoms, conditions, stage, hygiene, room). The 24 rules pattern-match case facts against domain facts (and, in R24, against another rule's result) to derive `diagnose/1`; as each rule fires, the engine records the matching facts and the rule id so the explanation can be rebuilt afterwards.

From symptoms to a recommendation: the inference pipeline



Figure 5: Inference pipeline — the six-step runtime flow from consultation input to explanation.

FastAPI turns the submitted form into case facts; SWI-Prolog matches them against the rule base and looks up the recommendation before the explanation is composed and returned as JSON.

Backward chaining. The natural mode: FastAPI issues the goal `findall(Problem, diagnose(Problem), Problems)`, and SWI-Prolog's SLD resolution works backward from each `diagnose(Problem)` clause, trying to prove its body against the asserted case facts. This directly answers "is Problem X present, and why?" and is used for the main per-consultation query.

Forward chaining. To demonstrate genuine data-driven reasoning (not just backward resolution), every rule is also written as `rule(RuleID, ConditionList, Conclusion)`, e.g. `rule(r1, [observed_symptom(long_stem), observed_symptom(small_cap), current_condition(co2_level, high)], high_co2_stress)`. A `forward_chain/0` predicate repeatedly scans `rule/3`, checks whether every condition in

ConditionList currently holds (asserted or already derived), and asserts the Conclusion the first time it is satisfied; it stops when a full pass derives nothing new (fixpoint). This lets the system start purely from the grower's facts and derive everything it can, independent of any single question — useful for an "explore all issues" report.

Explanation. Whichever mode is used, each successful match also asserts `evidence_for(Problem, Fact)` for every case fact consumed and `rule_fired(Problem, RuleID)` for the clause that fired. `explanation/2` composes these into a sentence, e.g. *"Because long_stem and small_cap were observed while co2_level is high, rule R1 concluded high_co2_stress."* `recommendation/2` is then looked up to fill the Management Recommendation field.

Inference example. The two traces below work through the same input — `long_stem`, `small_cap`, `co2_level = high` — once backward and once forward, to make concrete how the two strategies above actually execute.

Backward chaining (goal-driven): FastAPI asks whether `diagnose(high_co2_stress)` holds, and SWI-Prolog's SLD resolution works backward through R1's body against the case facts:

Goal: `diagnose(high_co2_stress)`

```
|
Check R1's body
|
long_stem observed?    yes
small_cap observed?    yes
co2_level = high?      yes
|
R1 succeeds -> high_co2_stress
|
evidence_for/2, rule_fired/2 recorded -> explanation, recommendation
```

Forward chaining (data-driven): starting only from the asserted facts, `forward_chain/0` scans every rule/3 fact until a full pass derives nothing new:

Asserted: `long_stem`, `small_cap`, `co2_level(high)`

```
|
Scan rule/3: r1's ConditionList satisfied?    yes
|
Assert high_co2_stress (r1's Conclusion)
|
Scan again: any other rule's ConditionList now satisfied?    no
|
Fixpoint reached -> {high_co2_stress}
```

Both traces reach the same conclusion from the same facts, which is expected: backward chaining asks a targeted question and finds the rule that answers it, while forward chaining derives everything the facts support and simply happens to include it. This exact scenario is also exercised live in Testing (#1).

To prove SWI-Prolog itself is doing the reasoning (not Python producing the answer), the following was run directly against `prolog/facts.pl`, `rules.pl` and `engine.pl` via a small helper script (`prolog/demo_trace.pl`) — bypassing FastAPI entirely:

```
swipl -q -g main -t halt demo_trace.pl
```

Rule R1 fired

Evidence:

- long_stem was observed
- small_cap was observed
- co2_level is high

Conclusion:

high_co2_stress

Figure 6: Prolog inference and reasoning trace — `demo_trace.pl` run directly against the knowledge base, bypassing FastAPI entirely.

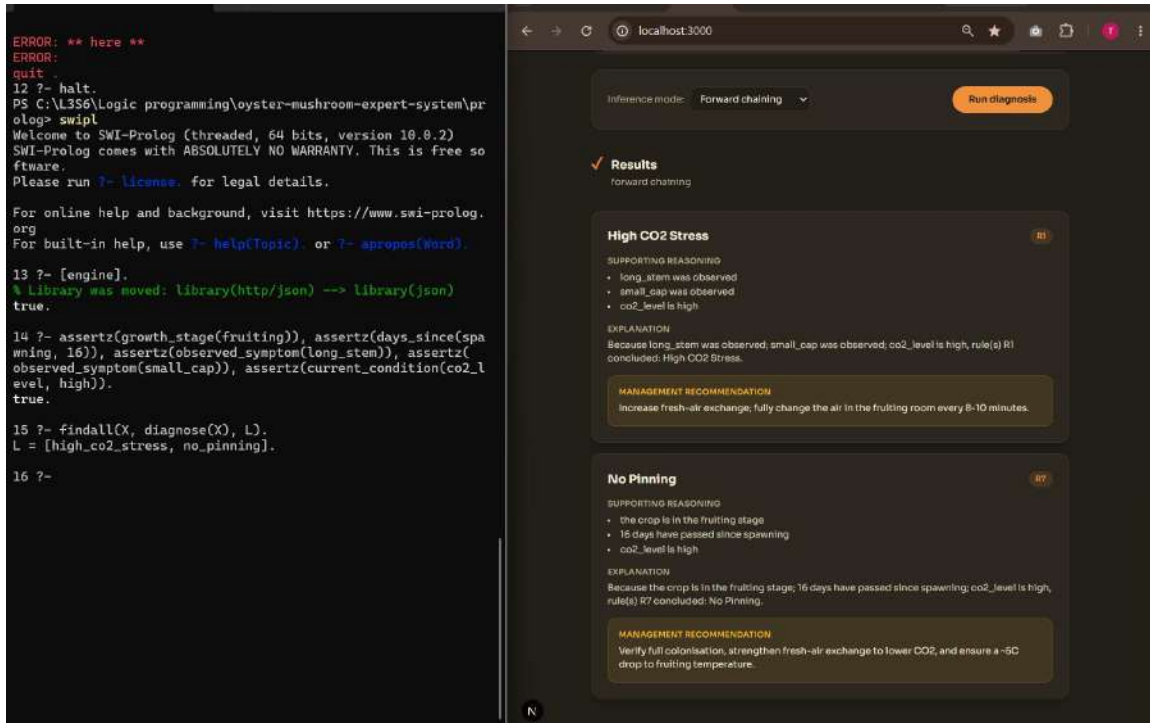


Figure 7: Same input, same answer on both ends — the SWI-Prolog top level (`findall/3`) and the browser results view agree on `[high_co2_stress, no_pinning]` (rules R1 and R7).

Input and Output Representation

Input (Next.js form → FastAPI request body). The grower fills in the crop's stage, days since spawning, a symptom checklist and estimated/measured conditions:

```
{
  "growth_stage": "fruiting",
  "days_since_spawning": 16,
  "symptoms": ["long_stem", "small_cap"],
  "conditions": {
    "co2_level": "high",
    "humidity_percent": 70,
    "light_hours": 6,
    "temperature_drop": "none"
  },
  "hygiene": { "substrate_sterilization": "unknown" },
}
```

```
"room": { "window_screens": "present", "bag_spacing": "normal" }
}
```

Each field under conditions accepts either a number ("co2_ppm": 1400, "humidity_percent": 70) or a category ("co2_level": "high", "humidity_level": "dry") — whichever the grower can supply. FastAPI normalises whichever form arrives into the case facts the rules expect: a numeric humidity/light reading is asserted as-is for the threshold rules (R5, R6), while a category is mapped to a representative value from optimal_condition/5 before asserting. For CO2, both forms are asserted directly, since the rule base already reasons over either the category (R1) or the ppm reading (R2).

FastAPI → Prolog. FastAPI validates this JSON (Pydantic model) and translates each field into assertz/1 calls before running the query, e.g. assertz(growth_stage(fruiting)), assertz(days_since(spawning, 16)), assertz(observed_symptom(long_stem)), assertz(observed_symptom(small_cap)), assertz(current_condition(co2_level, high)), assertz(current_condition(humidity_percent, 70)). Each request runs in a fresh Prolog module (or the case facts are retracted at the end) so consultations never leak into one another.

Output (Prolog → FastAPI → UI). FastAPI collects every diagnose/1 solution together with its evidence, firing rule and recommendation, and returns:

```
{
  "diagnoses": [
    {
      "problem": "high_co2_stress",
      "evidence": [
        "observed_symptom(long_stem)",
        "observed_symptom(small_cap)",
        "current_condition(co2_level, high)"
      ],
      "rules_applied": ["R1"],
      "explanation": "Long stems with small caps while CO2 is reported high match the classic legginess pattern of CO2 buildup from inadequate fresh-air exchange.",
      "recommendation": "Increase fresh-air exchange; fully change the air in the fruiting room every 8-10 minutes."
    }
  ]
}
```

The React UI renders one card per diagnosis, each showing the four required outputs (problem, reasoning, rules applied, recommendation) so a grower can see not just *what* the system concluded but *why*.

Testing

Two automated test suites exercise the system: tests/test_rules.pl (40 SWI-Prolog PIUnit tests, one or more per rule, run directly against the knowledge base) and tests/test_api.py (16 pytest integration tests against the real FastAPI + Prolog subprocess, no mocking). All 56 pass. Run them with:

```
swipl -q -g run_tests -t halt tests/test_rules.pl
PYTHONPATH=backends python3 -m pytest tests/test_api.py -v
```

The table below is a representative sample (the full suites cover every rule); "Actual output" reflects what the running system returned when this table was written.

#	Scenario (input)	Expected	Actual	Rule(s)	Result
---	------------------	----------	--------	---------	--------

#	Scenario (input)	Expected	Actual	Rule(s)	Result
1	long_stem, small_cap observed; co2_level=high	high_co2_stress	high_co2_stress, evidence = the 3 facts, explanation cites R1	R1	Pass
2	growth_stage=fruiting, co2_ppm=1500 (numeric only, no category)	high_co2_stress via the numeric- threshold rule	high_co2_stress via R2 (1500 > 1200 ppm threshold)	R2	Pass
3	long_stem, pale_cap; light_hours=4 (fruiting)	insufficient_ligh t, distinguished from CO2 stress by pallor	insufficient_light	R3	Pass
4	dry_cracked_cap; humidity_percent=60 (fruiting)	low_humidity_s tress	low_humidity_stress	R5	Pass
5	growth_stage=fruiting, days_since_spawning=16, no pins, co2_level=high	no_pinning (CO2 branch)	no_pinning	R7	Pass
6	Same as #5, but colonisation=incomplete instead of high CO2	no_pinning (colonisation branch)	no_pinning	R8	Pass
7	growth_stage=fruiting, temperature_drop=none	delayed_pinning	delayed_pinning	R9	Pass
8	green_patches_on_substrat e; days_since_spawning=12	green_mold_co ntamination (within 10–15 day onset window)	green_mold_contamination	R14	Pass
9	green_patches_on_substrat e; days_since_spawning=25 (outside window)	No green-mold diagnosis	No diagnosis returned	R14	Pass
10	brown_spots_on_cap; humidity_percent=95	bacterial_brown _blotch	bacterial_brown_blotch	R16	Pass
11	hygiene.substrate_steriliza tion=poor	general_contami nation	general_contamination	R18	Pass
12	insects_present; room.window_screens=ab sent	pest_infestation	pest_infestation	R21	Pass

#	Scenario (input)	Expected	Actual	Rule(s)	Result
13	thin_stipe, delayed_fruiting, abnormal_shape, nothing else	possible_viral_i nfection (last- resort, other causes absent)	possible_viral_infection	R24	Pass
14	Same as #13, plus long_stem, small_cap, co2_level=high	R24 must not fire once CO2 stress explains the case	Only high_co2_stress returned; R24 suppressed	R1, R24 (negate d)	Pass
15	No symptoms or conditions at all	Empty diagnosis list, no false positive	{"diagnoses": []}	—	Pass
16	Scenario #1 re-run with "mode": "forward"	Same conclusions as backward chaining, reached by fixpoint iteration instead of SLD resolution	Identical diagnoses, via forward_chain/0	R1, R7	Pass
17	growth_stage="not_a_real _stage" (invalid input)	Request rejected before reaching Prolog	HTTP 422 validation error	—	Pass
18	Two consecutive unrelated consultations	Case facts from consultation 1 must not leak into consultation 2	Consultation 2 returned no diagnoses despite consultation 1's symptoms	—	Pass

Explanation testing. Every scenario above was also checked for the *content* of its explanation, not just the diagnosis id: each returned explanation string names every fact that was matched (e.g. *"Because long_stem was observed; small_cap was observed; co2_level is high, rule(s) R1 concluded: High CO2 Stress."*) and the evidence list matches exactly the case facts the rule consumed — verified by tests/test_api.py::test_consult_high_co2_stress and the equivalent PIUnit tests.

Knowledge Engineering Traceability

The table below follows every acquired piece of knowledge from its source, through the Knowledge Acquisition entry and the Prolog rule it justifies, to the automated test that exercises that rule. TC01–TC24

label the primary tests/test_rules.pl PIUnit test case for each of R1–R24 (several rules have more than one covering test — see Testing above).

Source knowledge	KA ID	Prolog representation	Rule	Test
Poor fresh-air exchange (high CO ₂) causes long stems + small caps	KA09	current_condition(c o2_level, high)	R1	TC01
Fruiting CO ₂ threshold $\approx$ 1200 ppm	KA08	co2_threshold(fruiting, 1200)	R2	TC02
Insufficient light causes long, pale, leggy stems	KA10, KA11	photoperiod(fruiting, Min, _)	R3	TC03
Oyster mushroom needs 12–18 h/day indirect light during fruiting	KA11	Hours $\leq$ 4	R4	TC04
Fruiting relative humidity should be 80–90%	KA07, KA15	optimal_condition(fruiting, humidity, Min, _, percent)	R5	TC05
Dry air dries out/curles pins below $\approx$ 70% RH	KA15	H < 70	R6	TC06
No pins after $\approx$ 14 days $\Rightarrow$ CO ₂ still too high	KA12	D $\geq$ 14, current_condition(c o2_level, high)	R7	TC07
No pins after $\approx$ 14 days $\Rightarrow$ incomplete colonisation	KA12	current_condition(colonisation, incomplete)	R8	TC08
A $\approx$ 5°C drop from colonisation temperature triggers pinning	KA06	current_condition(temperature_drop, none)	R9	TC09
CO ₂ fluctuation aborts formed pins	KA14	current_condition(c o2_level, fluctuating)	R10	TC10

Source knowledge	KA ID	Prolog representation	Rule	Test
Inconsistent humidity aborts formed pins	KA14	$H < 75$	R11	TC11
Airflow without matching humidity dries pins before capping	KA13	current_condition(fresh_air_exchange, good), $H < 70$	R12	TC12
Overcrowded bags reduce air circulation	KA21	room_condition(bag_spacing, crowded)	R13	TC13
Green mould onset 10–15 days after spawning	KA16	$D \geq 10$, $D \leq 15$	R14	TC14
Green mould caused by <i>Trichoderma</i> spp.	KA16	disease_cause(green_mold_contamination, trichoderma_spp)	R15	TC15
Bacterial brown blotch favoured by high humidity	KA18	$H > 90$	R16	TC16
Brown blotch linked to free water on caps	KA18	current_condition(free_water_on_caps, present)	R17	TC17
Poor substrate sterilisation causes contamination	KA17	hygiene_practice(substrate_sterilization, poor)	R18	TC18
Poor equipment sterilisation causes contamination	KA17	hygiene_practice(equipment_sterilization, poor)	R19	TC19
Spent substrate near the growing room causes contamination	KA17	room_condition(spent_substrate_disposal, near_growing_room)	R20	TC20
Pests enter through unscreened doors/windows	KA19	room_condition(window_screens, absent)	R21	TC21

Source knowledge	KA ID	Prolog representation	Rule	Test
Old spawn causes weak/absent colonisation	KA20	current_condition(s pawn_age, old)	R22	TC22
Unverified spawn source causes patchy colonisation	KA20	current_condition(s pawn_source, unverified)	R23	TC23
Viral infection causes delayed fruiting, thin stipe, abnormal shape	KA22	\+ diagnose(high_co2_ stress), \+ diagnose(insufficient_ light)	R24	TC24

This traceability ensures that each diagnostic rule can be traced from its original domain source through knowledge acquisition and Prolog implementation to a corresponding test case.

Assumptions

- The system assumes that the user is cultivating *Pleurotus* (oyster mushroom).
- User-reported symptoms are assumed to be reasonably accurate.
- Environmental measurements are assumed to be taken using reasonably functioning measuring devices where numerical values are provided.
- The system uses documented cultivation ranges as guidance rather than absolute biological limits.
- Multiple diagnoses may be returned when the same conditions support more than one rule.

Limitations

- The system provides possible cultivation problems, not guaranteed diagnoses.
- The accuracy depends on the correctness of user-provided conditions and symptoms.
- The current knowledge base focuses on common oyster mushroom cultivation problems and does not cover every possible disease or pest.
- Some symptoms, such as long stems or poor fruiting, can have multiple possible causes.
- The system uses a fixed rule base and therefore cannot automatically learn new cultivation knowledge.
- Environmental thresholds may vary according to oyster mushroom variety and local cultivation conditions.
- The system has not been validated against a large dataset of real farms or expert diagnoses; the 56 automated tests (see Testing) verify that the software and rules behave as intended, not that the rules are universally correct in real mushroom farms.

Disclaimer

This expert system provides rule-based cultivation guidance based on the information entered by the user. Results are possible explanations, not guaranteed diagnoses. For serious or persistent contamination problems, consult a qualified mushroom cultivation specialist.

User Manual

Required software/tools

- SWI-Prolog 9.x (swipl on PATH)
- Python 3.10+ with pip
- Node.js 20.9+ with npm

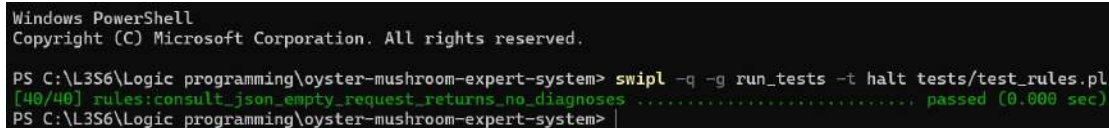
Project layout

```
oyster-expert-system/
  prolog/   facts.pl, rules.pl, engine.pl, run_consultation.pl
  backend/  FastAPI app (main.py, models.py, prolog_bridge.py, vocabulary.py)
  frontend/ Next.js + React + Tailwind UI
  tests/    test_rules.pl (PLUnit), test_api.py (pytest)
```

Installation/setup

1. Install SWI-Prolog: `sudo apt-get install swi-prolog-nox` (Linux) or download from swi-prolog.org (Windows/macOS). Verify with `swipl --version`.
2. Backend dependencies: `cd backend && pip install -r requirements.txt`.
3. Frontend dependencies: `cd frontend && npm install`.
4. Copy `frontend/.env.local.example` to `frontend/.env.local` (default already points at `http://localhost:8000`).

How to start/run the system

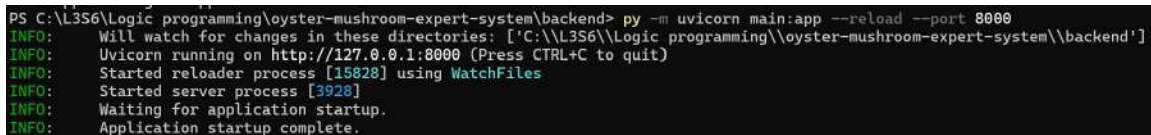


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\L3S6\Logic programming\oyster-mushroom-expert-system> swipl -q -g run_tests -t halt tests/test_rules.pl
[40/40] rules:consult_json_empty_request_returns_no_diagnoses ..... passed (0.000 sec)
PS C:\L3S6\Logic programming\oyster-mushroom-expert-system> |
```

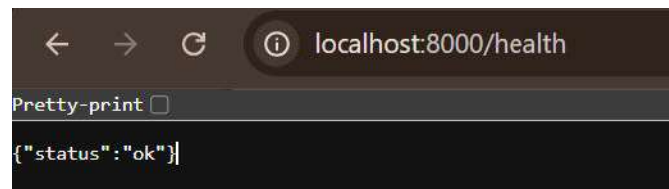
Figure 8: The PLUnit test suite passing — 40/40 tests (tests/test_rules.pl).

1. Start the backend from the project root: `cd backend && python3 -m uvicorn main:app --reload --port 8000`. Confirm with `curl http://localhost:8000/health` → `{"status": "ok"}`.



```
PS C:\L3S6\Logic programming\oyster-mushroom-expert-system\backend> py -m uvicorn main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\L3S6\Logic programming\oyster-mushroom-expert-system\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15828] using WatchFiles
INFO: Started server process [3928]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figure 9: The FastAPI backend starting with Uvicorn.



```
localhost:8000/health
Pretty-print ☐
{"status": "ok"}
```

Figure 10: GET /health returning `{"status": "ok"}`.

2. Start the frontend in a second terminal: `cd frontend && npm run dev`.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\L3S6\Logic programming\oyster-mushroom-expert-system> cd frontend
PS C:\L3S6\Logic programming\oyster-mushroom-expert-system\frontend> npm run dev

> frontend@0.1.0 dev
> next dev

▲ Next.js 16.3.6 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.56.1:3000
- Environments: .env.local
✓ Ready in 7.3s
✓ Running next.config.ts took 1191ms

```

Figure 11: The Next.js frontend starting in development mode.

3. Open <http://localhost:3000> in a browser.

EXPERT SYSTEM

Oyster Mushroom Cultivation Diagnosis

A rule-based expert system for diagnosing common *Pleurotus* species cultivation problems. Fill in what you can observe or measure. Fields are optional.

01 Growth details

Growth stage: Fruiting (dropdown) Days since spawning: e.g. 16 (input)

02 Observed symptoms

Tap all that apply

Long stem (selected) Small / underdeveloped cap (selected) Pale cap colour Brittle cap

Dry, cracked cap Pins drying out Pins aborting before maturity

Long, thin stem No cap forming Poor air circulation

Green patches on substrate (selected) Green sporulation Brown spots on cap

Insects present No mycelium growth Patchy colonisation Thin stipe

Delayed fruiting-body formation Abnormal fruiting-body shape

03 Growing conditions

Figure 12: The consultation form's homepage, freshly loaded.

No separate step is needed to "start" Prolog: FastAPI invokes `swipl` as a short-lived subprocess for every consultation (see `backend/prolog_bridge.py`), so SWI-Prolog only needs to be installed and on PATH.

localhost:3000

01 Growth details

Growth stage: Fruiting (dropdown)
Days since spawning: e.g. 16 (input field)

02 Observed symptoms

Tap all that apply

Selected symptoms (highlighted in orange): Long stem, Small / underdeveloped cap, Green patches on substrate.

Other symptoms: Pale cap colour, Brittle cap, Dry, cracked cap, Pins drying out, Pins aborting before maturity, Long, thin stem, No cap forming, Poor air circulation, Green sporulation, Brown spots on cap, Insects present, No mycelium growth, Patchy colonisation, Thin stipe, Delayed fruiting-body formation, Abnormal fruiting-body shape.

03 Growing conditions

CO2 level (category): high (dropdown menu open, showing options: high, -- not reported --, high, fluctuating, normal, -- not reported --)

CO2 reading (ppm) (ppm): 0 (input field)

Relative humidity (%) (%): not measured (input field)

Fresh-air exchange quality: -- not reported -- (dropdown)

Substrate colonisation: -- not reported -- (dropdown)

Spawn age: -- not reported -- (dropdown)

Spawn source: -- not reported -- (dropdown)

Free water sitting on caps: -- not reported -- (dropdown)

Figure 13: Observed symptoms ticked and the CO₂ level category dropdown open on "high".

Temperature drop offered to trigger pinning: -- not reported -- (dropdown)

Fresh-air exchange quality: -- not reported -- (dropdown)

Substrate colonisation: -- not reported -- (dropdown)

Spawn age: -- not reported -- (dropdown)

Spawn source: -- not reported -- (dropdown)

Free water sitting on caps: -- not reported -- (dropdown)

04 Hygiene & room conditions

HYGIENE

Substrate sterilisation: -- not reported -- (dropdown)

Equipment sterilisation: -- not reported -- (dropdown)

ROOM CONDITIONS

Bag/block spacing: -- not reported -- (dropdown)

Window/door wire mesh screens: -- not reported -- (dropdown)

Spent substrate disposal: -- not reported -- (dropdown)

Inference mode: Backward chaining (dropdown menu open, showing options: Backward chaining, Forward chaining)

Run diagnosis (button)

Figure 14: The inference-mode dropdown, showing the Backward chaining / Forward chaining choice.

Figure 15: A populated result card after clicking Run diagnosis — High CO2 Stress, rule R1 (backward chaining).

How to interact with the UI

1. Optionally set the growth stage and days since spawning.
2. Tick any observed symptoms (checkboxes).
3. Fill in any growing conditions, hygiene and room fields you can report — every field is optional; leave a field blank if it wasn't measured. CO2 in particular can be given either as a category (High/Fluctuating/Normal) or a ppm reading, whichever you have.
4. Choose the inference mode (Backward chaining is the default and answers "what does the evidence prove"; Forward chaining derives everything reachable from the facts via a rule-firing fixpoint — both should agree on the same input).
5. Click **Run diagnosis**. Each result card shows the problem, the supporting evidence, the rule(s) applied, a plain-language explanation, and a management recommendation.

Example input (typed into the form, or sent directly to the API)

```
POST http://localhost:8000/consult
{
  "growth_stage": "fruiting",
  "days_since_spawning": 16,
  "symptoms": ["long_stem", "small_cap"],
  "conditions": { "co2_level": "high" }
}
```

Example output

```
{
  "mode": "backward",
  "diagnoses": [
    {
      "problem": "high_co2_stress",
```

```

"label": "High CO2 Stress",
"evidence": ["long_stem was observed", "small_cap was observed", "co2_level is high"],
"rules_applied": ["R1"],
"explanation": "Because long_stem was observed; small_cap was observed; co2_level is high, rule(s)
R1 concluded: High CO2 Stress.",
"recommendation": "Increase fresh-air exchange; fully change the air in the fruiting room every 8-10
minutes."
},
{
  "problem": "no_pinning",
  "...": "(a second, independent diagnosis from the same input; see Testing
#5)"
}
]
}

```

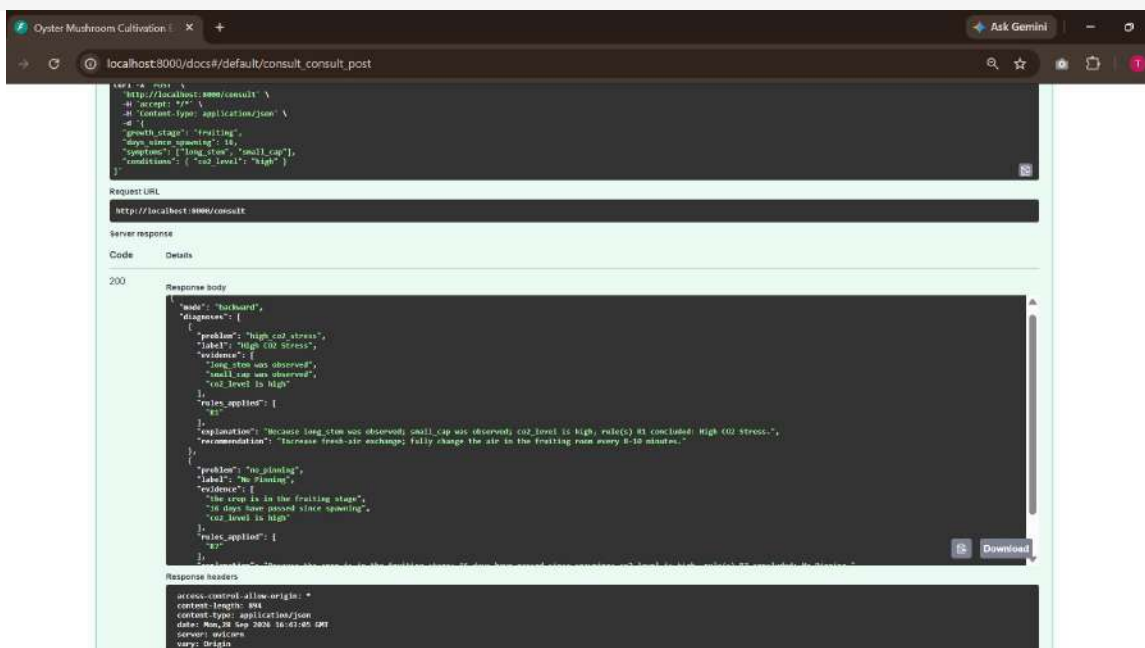


Figure 16: The FastAPI Swagger UI (/docs) executing POST /consult and returning both diagnoses shown in the example above.

Other configuration. The frontend reads the backend's address from `NEXT_PUBLIC_API_URL` (frontend/.env.local); change it if the backend runs on a different host/port. The backend allows all CORS origins by default for local development (backend/main.py); tighten this before any real deployment. No database or API keys are required — the entire knowledge base lives in the `prolog/` files.

Conclusion

This project implements a rule-based expert system for diagnosing common cultivation problems in oyster mushroom (*Pleurotus* spp.) farming, built in SWI-Prolog with a FastAPI backend and a Next.js/React front end. Its knowledge base holds 32 domain facts and 24 diagnostic rules, sourced from HORDI/Department of Agriculture Sri Lanka, University of Florida IFAS Extension and peer-reviewed *Pleurotus* research, and reasons over them using both backward chaining (SLD resolution via `diagnose/1`) and forward chaining (a fixpoint rule-firing loop via `forward_chain/0`). An explanation facility accompanies every diagnosis with the specific evidence and rule(s) that produced it, so a grower sees not just what was concluded but why. The system has been validated with 40 SWI-Prolog PLUnit tests and 16 pytest API tests, all passing, covering every rule and the no-diagnosis case. Overall, it gives Sri Lankan

oyster mushroom growers a fast, transparent, evidence-based first opinion on what is going wrong in their growing room and what to do about it.

Sources

Sources are grouped by tier: Tier 1 is institutional (HORDI / Department of Agriculture Sri Lanka), university extension services, and peer-reviewed research, which the required rules are prioritised on; Tier 2 is technical cultivation guides, used only for practical troubleshooting detail the academic literature does not cover. The [S1]–[S16] tags are the same ones cited in the Rule Source Table.

Tier 1 — HORDI / Department of Agriculture, university extension services and peer-reviewed research

- [S1] Department of Agriculture Sri Lanka, Horticultural Crops Research and Development Institute (HORDI), "Mushroom Crop." [Online]. Available: <https://doa.gov.lk/hordi-crop-mushroom/>
- [S2] Department of Agriculture Sri Lanka, Horticultural Crops Research and Development Institute (HORDI), "Mushroom Varieties." [Online]. Available: <https://doa.gov.lk/hordi-variety-mushroom/>
- [S3] K. Hameed, V. T. Cotter, H.-L. Liao, and C.-M. Hsu, "D.I.Y. FunGuide: Grow Your Own Oyster Mushrooms at Home," UF/IFAS Extension, SL448/SS662, 2018, revised 2025. doi: 10.32473/edis-ss662-2018. [Online]. Available: <https://ask.ifas.ufl.edu/publication/SS662>
- [S4] H.-L. Liao, C.-M. Hsu, K. Hameed, and V. T. Cotter, "Isolation of Mother Cultures and Preparation of Spawn for Oyster Mushroom Cultivation," UF/IFAS Extension, SL449/SS663. [Online]. Available: <https://ask.ifas.ufl.edu/publication/SS663>
- [S5] A. Burtch and M. E. Smith, "Oyster Mushrooms (*Pleurotus* species) of Florida," UF/IFAS Extension, PP384, 2025. doi: 10.32473/edis-pp384-2025. [Online]. Available: <https://ask.ifas.ufl.edu/publication/PP384>
- [S6] "Diseases and pests noxious to *Pleurotus* spp. mushroom crops," *Revista Argentina de Microbiología*, vol. 50, no. 2, pp. 216–226, 2018. doi: 10.1016/j.ram.2017.08.007. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0325754117301505>
- [S8] Y.-B. Yun, K.-H. Cho, and Y.-K. Kim, "Inhibition of Tolaasin Cytotoxicity Causing Brown Blotch Disease in Cultivated Mushrooms Using Tolaasin Inhibitory Factors," *Toxins*, vol. 15, no. 1, p. 66, 2023. doi: 10.3390/toxins15010066. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9867037/>
- [S9] H. Aghajani, E. Bari, M. Bahmani, M. Humar, M. A. T. Ghanbary, D. D. Nicholas, and E. Zahedian, "Influence of relative humidity and temperature on cultivation of *Pleurotus* species," *Maderas. Ciencia y Tecnología*, vol. 20, no. 4, pp. 571–578, 2018. doi: 10.4067/S0718-221X2018005004501. [Online]. Available: <https://www.scielo.cl/pdf/maderas/v20n4/0718-221X-maderas-04501.pdf>
- [S10] G. Bekiaris, C. S. Pappas, A. Mastrogianis, L. Lachouvaris, P. A. Tarantilis, and G. I. Zervakis, "Optimizing *Pleurotus ostreatus* Mushroom Cultivation on Various Agro-Industrial By-Products—Development of a Process Analytical Technology Tool for Predicting Biological Efficiency," *Fermentation*, vol. 11, no. 10, p. 555, 2025. doi: 10.3390/fermentation11100555. [Online]. Available: <https://www.mdpi.com/2311-5637/11/10/555>
- [S16] I. J. A. Ruhunuge, M. K. I. Malhara, J. P. Kirthisinghe, and E. M. Wimalasiri, "Animal-Based Organic Substrate for Oyster Mushrooms (*Pleurotus ostreatus*) Cultivation in Sri Lanka," *International Symposium of Rajarata University 2021 (ISymRU 2021)*, 2021. [Online]. Available: https://www.researchgate.net/publication/358396460_Animal-

[Based Organic Substrate for Oyster Mushrooms Pleurotus ostreatus Cultivation in Sri Lanka](#)

Tier 2 — Technical cultivation guides (practical troubleshooting detail not found in the academic literature)

- [S7] N. O. Devi Nongthombam, "Common Diseases of Oyster Mushroom and Its Management," AgriCos e-Newsletter, vol. 2, no. 10, Article 04, 2021. [Online]. Available: https://www.agricosemagazine.com/_files/ugd/93e822_9daf7ec7e34e44d59f354992c63c50ac.pdf
- [S11] Fungi Ally, "Fix Oyster Mushroom Growing Problems for Better Yields," 2021. [Online]. Available: <https://www.fungially.com/blogs/growing-mushrooms/fix-oyster-mushroom-growing-problems-for-better-yields>
- [S12] Oyster-Mushroom.com, "Common Problems with Growing Oyster Mushrooms." [Online]. Available: <https://oyster-mushroom.com/common-problems-with-growing-oyster-mushrooms/>
- [S13] H. De Los Reyes, "13 Oyster Mushroom Growing Problems & How To Fix It," BootstrapBee, updated May 12, 2023. [Online]. Available: <https://bootstrapbee.com/mushrooms/13-oyster-mushroom-growing-problems-how-to-fix-it>
- [S14] Shroom Crafter, "Why Mushroom Fruiting Fails Without Proper Airflow," May 30, 2026. [Online]. Available: <https://shroomcrafter.org/2026/05/30/why-mushroom-fruiting-fails-without-airflow/>
- [S15] usemycro, "How to Get Mushrooms to Pin: Triggers, Conditions & Troubleshooting." [Online]. Available: <https://usemycro.com/guides/mushroom-pinning.html>